CS201 Final

19 August 2024

Olcay Taner YILDIZ

***Abstract*— Write only ONE function per question. Do not check the input, assume that the input is correct. Unless noted, (i) the input data structure is not empty, (ii) time complexity of the algorithm does not matter, (iii) you can not use extra data structures including arrays, (iv) you can use any function given in the data structure, (v) you can not modify the data structure contents.**

I. Question (Algorithm Analysis) (20 pts)

```
1  int algorithm1(int N){
2      if (N == 0)
3          return 0;
4      int sum = 0;
5      for (int i = 0; i < N; i++)
6          for (int j = 0; j < N; j++)
7              for (int k = 0; k < N; k++)
8                  sum++;
9      for (int i = 0; i < 2; i++)
10         for (int j = 0; j < 2; j++)
11             sum += algorithm1(N / 2) + algorithm1(N / 2);
12     return sum;
13 }
```

What is the time complexity of algorithm1? Solve the recurrence equation without the master theorem.

II. Question (Linked List)

Write the static method in **LinkedList** class

```
LinkedList difference(LinkedList list1,
                      LinkedList list2)
```

to find the difference of the elements in two sorted linked lists and return a new linked list. The resulting list should contain those elements that are in list1 but not in list2. Do not modify linked lists list1 and list2. Your method should run in $\mathcal{O}(N)$ time. Nodes in the resulting list should be new. You can not use any linked list methods except getters and setters.

III. Question (Tree)

Write a recursive method in **TreeNode** class

```
void accumulateLeaves(int* a, int& index)
```

that accumulates all leaf contents in the tree in array a, where the values in the array are will be sorted. Use and modify index to store the integers into correct positions. Your method should run in $\mathcal{O}(N)$ time. The function can be called like this:

```
int a[100];
int index = 0;
t.getRoot()->accumulateLeaves(a, index);
```

IV. Question (Graph)

Write the method in linked list implementation of **Graph** class

```
Graph merge(const Graph& g2, int v)
```

that returns a new graph formed by adding edges which exist in the original graph or g2. You may assume both graphs are weighted, if an edge exists in both graphs, add the resulting edge with the sum of their weights. You are not allowed to use any linked list methods.

V. Question (Sorting)

Suppose you are given three sorted arrays A, B, and C. Write a function in MergeSort class that returns the number of elements which are in A or B or C. Assume that all arrays have the same size and the last elements of all three arrays are the same. Your algorithm should run in $\mathcal{O}(N)$ time.

```
int inAorBorC(const int* A, const int* B,
              const int* C, int N)
```

VI. Question (Sorting)

Suppose you are given an array of N integers. Write a single pass algorithm in QuickSort class that puts the odd integers before the even integers similar to the partition algorithm in QuickSort.

```
void oddsBeforeEvens(int* A, int N)
```